

Chapter 5A:

Constraint Satisfaction Problems (CSP)

AMCS2104 Fundamentals of Artificial Intelligence

Content

- Constraint Satisfaction Problems
- Definition CSP
- CSP Formulation
- Solution to CSP
- Map Coloring Problem
- Constraint Graph
- Advantages of CSP Formulation
- Job Shop Scheduling Problem
- Variations on the CSP Formalism

Content

- Binarization of Constraints
- Absolute Constraints
- Preference Constraints
- Constraint Optimization Problem
- Constraint Propagation
- Local Consistency
- Node Consistency
- Arc Consistency

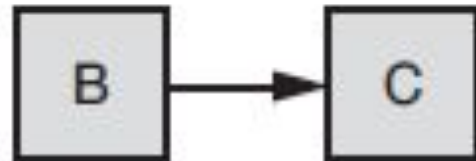
Search Problems

- Chapter 3, 4 and 5

- State: **Atomic representation**

- Solution

- A search problem is solved when a **goal state** is met (via a goal test).



Search Problems

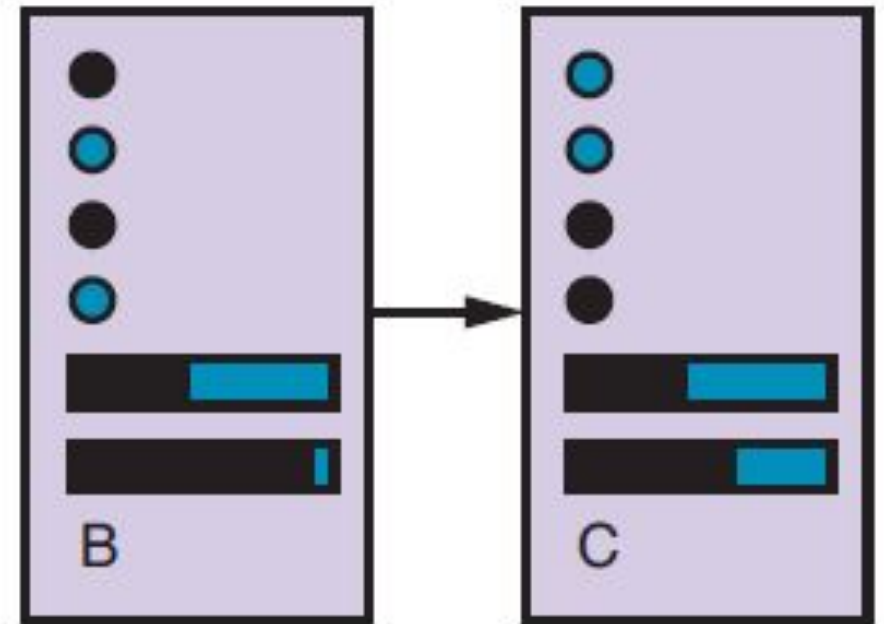
□ Chapter 6

□ State: **Factored representation**

- Each state has a set of variables holding different values.

□ Solution

- A **constraint satisfaction problem (CSP)** is solved when each variable has a value that **satisfies all the constraints on the variable**.



Constraint Satisfaction Problems

- CSP algorithms take the advantage of the structure of states and use **general-purpose heuristics** rather than **domain-specific heuristics** to enable the solution of complex problems.
- They eliminate large portions of the search space all at once by identifying variable/value combinations that violate the constraints

Domain-Specific Heuristics

- 2 examples of domain-specific heuristic are straight-line distance heuristic and Manhattan distance heuristic.

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Straight-line Distance

7	2	4
5		6
8	3	1

Manhattan distance

Defining CSP

- A constraint satisfaction problem consists of 3 components, X , D , and C :
 - X is a set of **variables**, $\{X_1, \dots, X_n\}$.
 - D is a set of **domains**, $\{D_1, \dots, D_n\}$, one for each variable.
 - Each domain D_x consists of a set of allowable values, $\{v_1, \dots, v_k\}$ for a variable X_i .
 - Different variables can have different domains of different sizes.
 - C is a set of **constraints**, $\{C_1, \dots, C_m\}$, that specify allowable combinations of values.

Defining CSP

- Each constraint C_i consists of a pair $\langle \text{scope}, \text{rel} \rangle$
 - **scope** is a tuple of variables that participate in the constraint.
 - **rel** is a **relation** that defines the values that those variables can take on.
- Suppose X_1, X_2 have the domain $\{1, 2, 3\}$, then constraint saying that X_1 must be greater than X_2 can be written as
 - An explicit list of all tuples of values that satisfy the constraint

$$\langle (X_1, X_2), \{(3,1), (3,2), (2,1)\} \rangle$$

- An abstract relation

$$\langle (X_1, X_2), X_1 > X_2 \rangle$$

Solution to CSP

□ CSPs deal with assignments of values to variables $\{X_i = v_i, X_j = v_j, X_k = v_k\}$.

□ **Consistent/legal assignment**

□ An assignment that does not violate any constraints.

□ **Complete assignment**

□ An assignment in which every variable is assigned.

□ **Partial assignment**

□ An assignment that assigns values to only some of the variables.

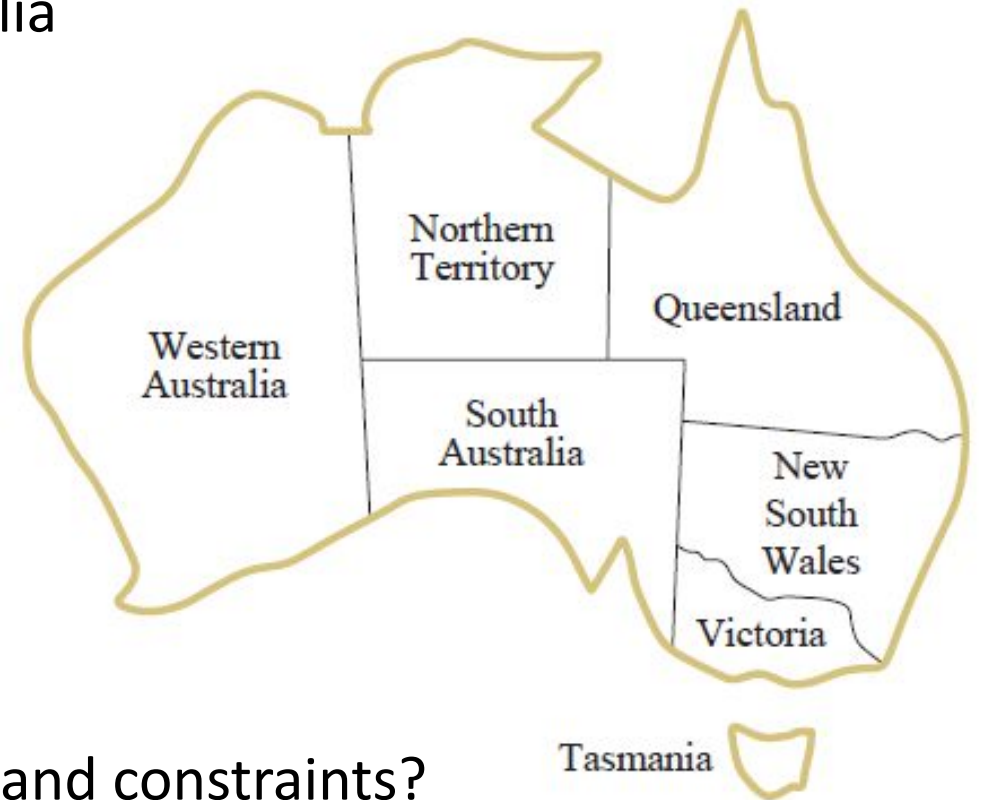
□ **Partial Solution**

□ A partial assignment that is consistent.

□ **A solution to a CSP is a consistent and complete assignment.**

Map Colouring Problem

- Suppose that, we are given a map of Australia showing each of its states and territories.
- We are given the task of **coloring each region either red, green, or blue** in such a way that **no neighboring regions have the same color**.
- **Question:** What are the variables, domains and constraints?



CSP Formulation

- To formulate this as a CSP, we define the variables to be the regions:

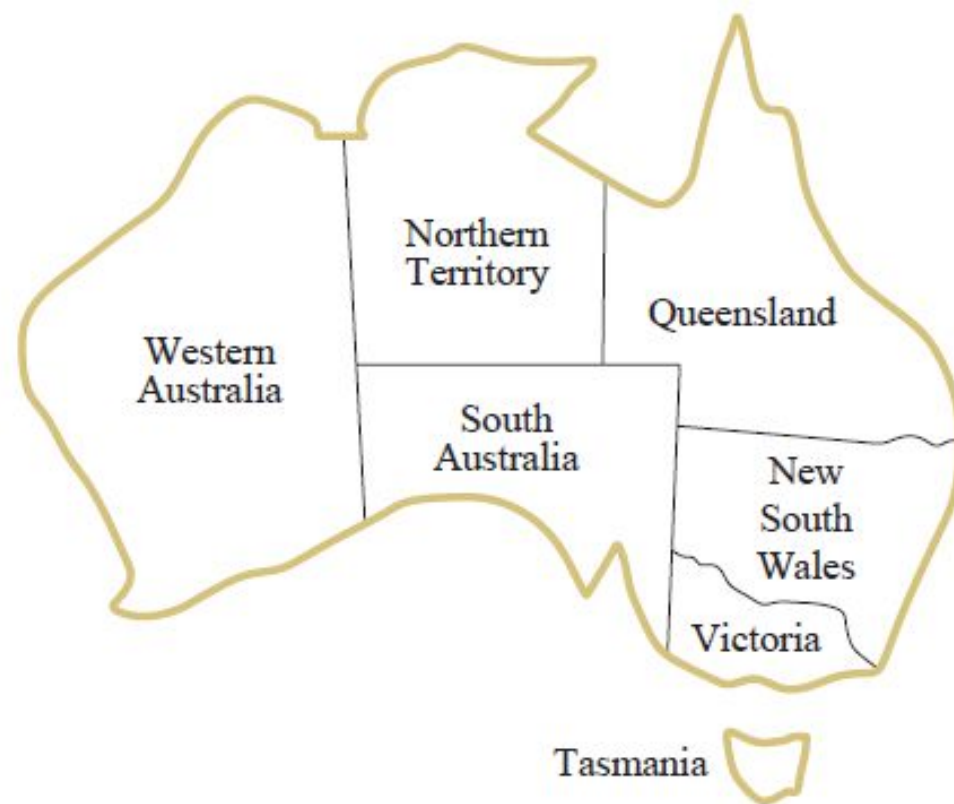
$$X = \{WA, NT, Q, NSW, V, SA, T\}$$

- The domain of each variable is the set:

$$D_i = \{\text{red, green, blue}\}$$

- The constraints require neighboring regions to have distinct colors:

$$C = \{SA \neq WA, SA \neq NT, SA \neq Q, SA \neq NSW, SA \neq V, WA \neq NT, NT \neq Q, Q \neq NSW, NSW \neq V\}$$



CSP Formulation

- An abstract relation is used to represent the constraints:

$$C = \{SA \neq WA, SA \neq NT, SA \neq Q, SA \neq NSW, SA \neq V, WA \neq NT, NT \neq Q, Q \neq NSW, NSW \neq V\}$$

- $SA \neq WA$ is a shortcut for $\langle (SA, WA), SA \neq WA \rangle$ where $SA \neq WA$ can be fully enumerated in turn as

$$\{(red, green), (red, blue), (green, red), (green, blue), (blue, red), (blue, green)\}$$

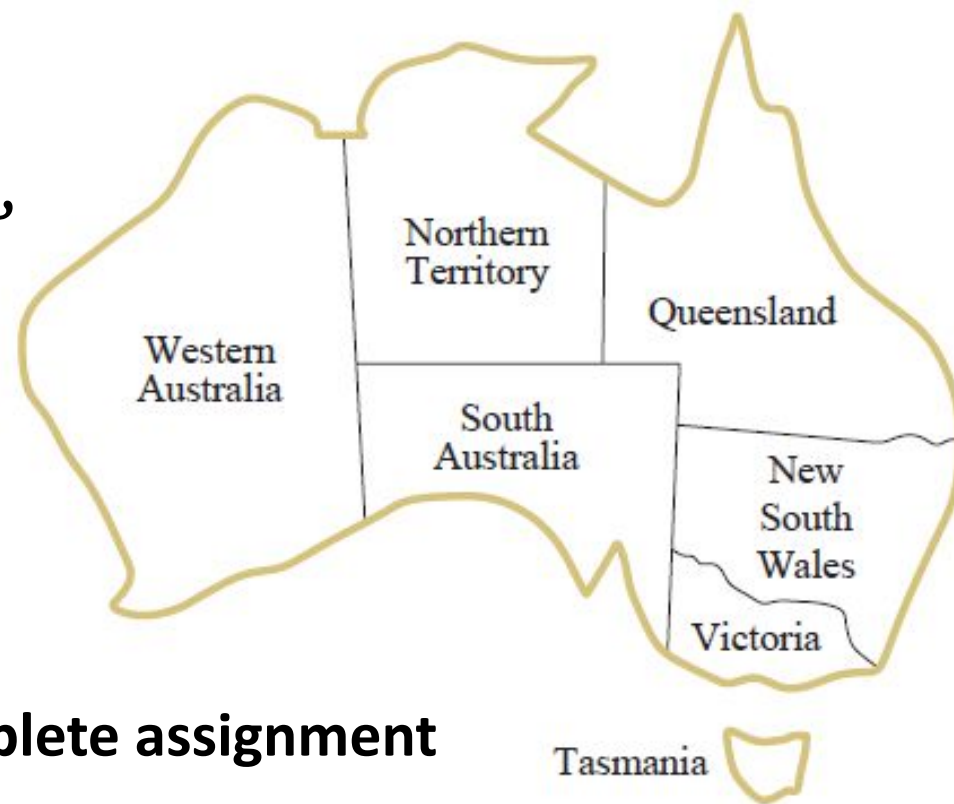
CSP Formulation

- One solution to this problem is

{WA=red, NT=green, Q=red, NSW=green, V=red, SA=blue, T=red}

- **Question:** How do we know if this is a solution?

- A **solution** to a CSP is a **consistent** and **complete assignment**



$C = \{SA \neq WA, SA \neq NT, SA \neq Q, SA \neq NSW, SA \neq V, WA \neq NT, NT \neq Q, Q \neq NSW, NSW \neq V\}$

CSP Formulation

□ There are many other possible solutions to this problem:

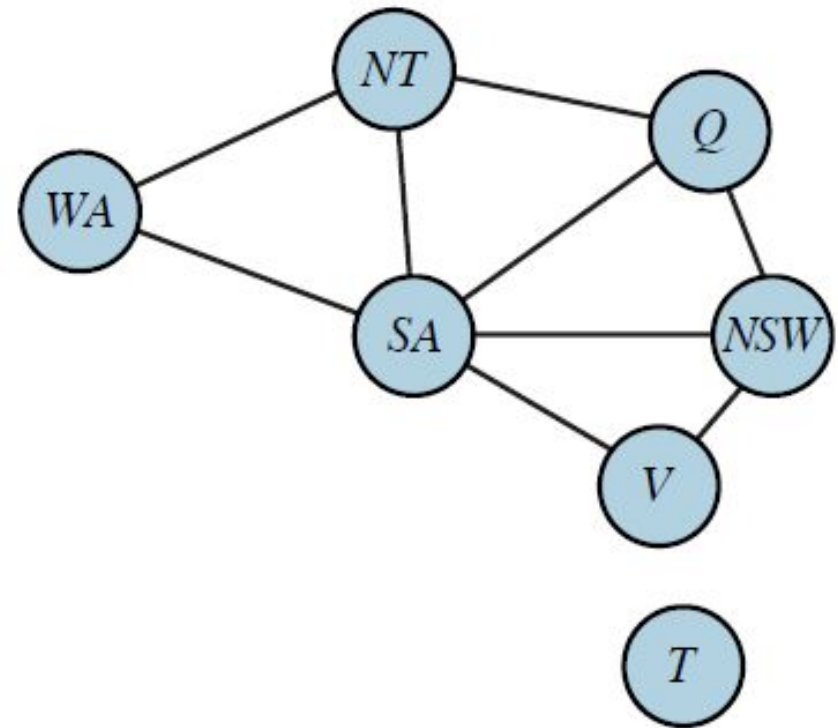
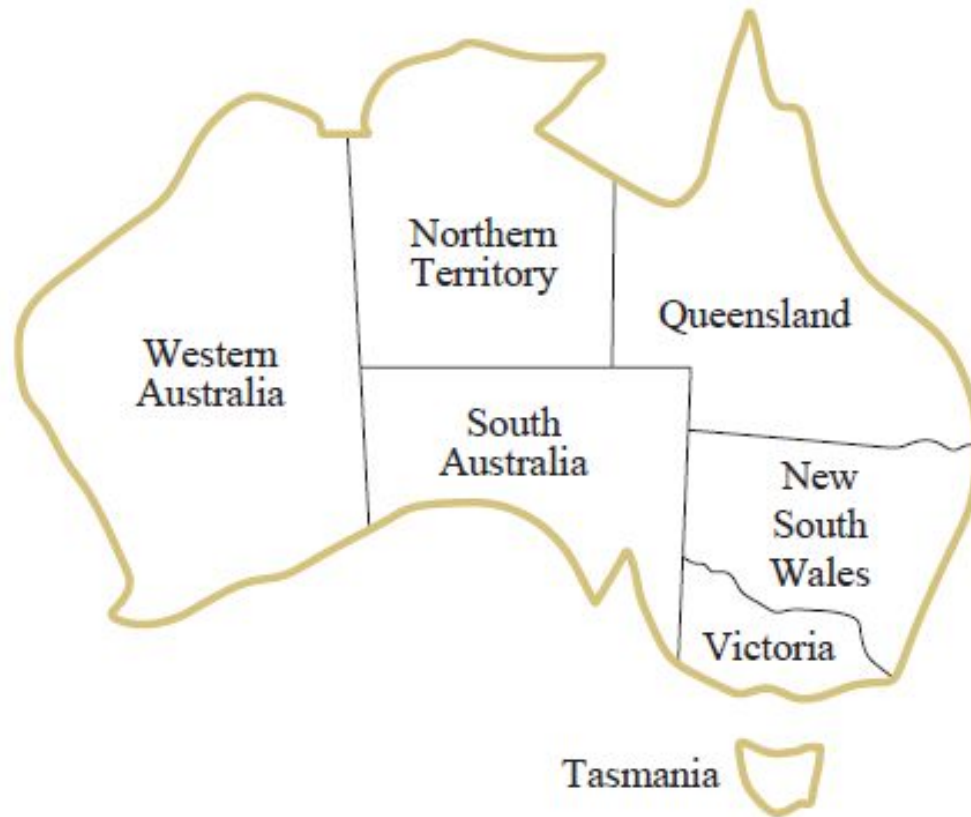
{WA=blue, NT=green, Q=blue, NSW=green, V=blue, SA=red, T=blue}

{WA=green, NT=blue, Q=green, NSW=blue, V=green, SA=red, T=red}

...

Constraint Graph

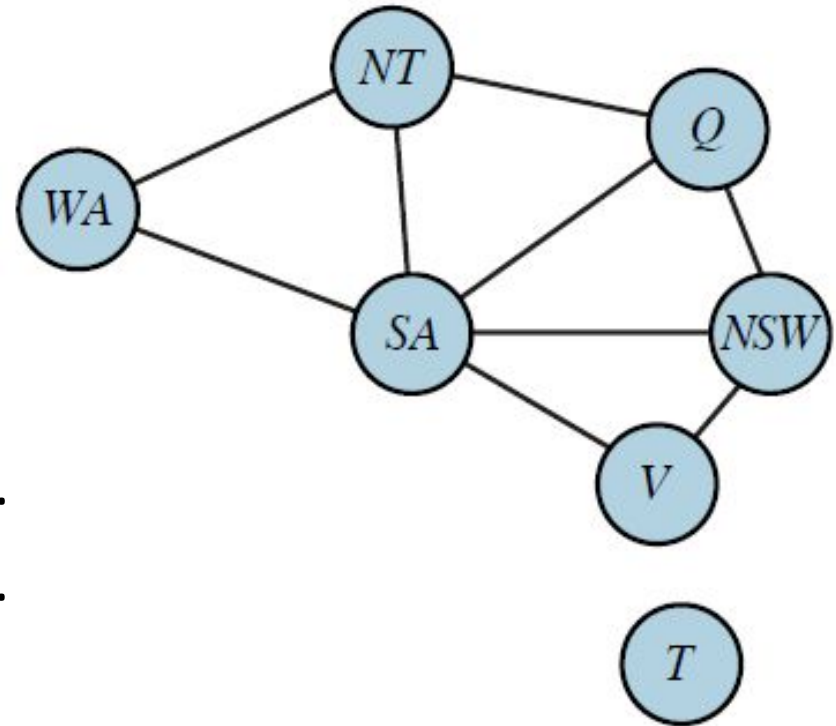
- The problem can be represented as a **constraint graph** for better visualization.



Why Formulate A Problem as CSP?

□ A **constraint graph** is a graphical representation of a CSP where:

- The **nodes** of the graph correspond to **variables** of the problem.
- An **edge** connects any two variables that participate in a **constraint**.



□ **Advantages**

- It is easy to formulate a problem as a CSP.
- Overall, **CSP solvers** are fast and efficient.

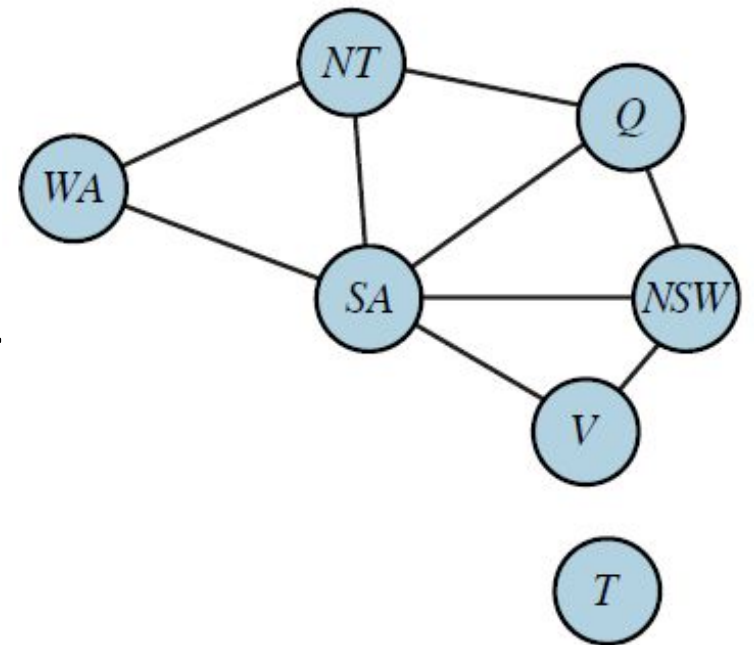
Why Formulate A Problem as CSP?

- Example

- Once the $\{SA = \text{blue}\}$ assignment is chosen, none of the five neighboring variables can take on the value blue.

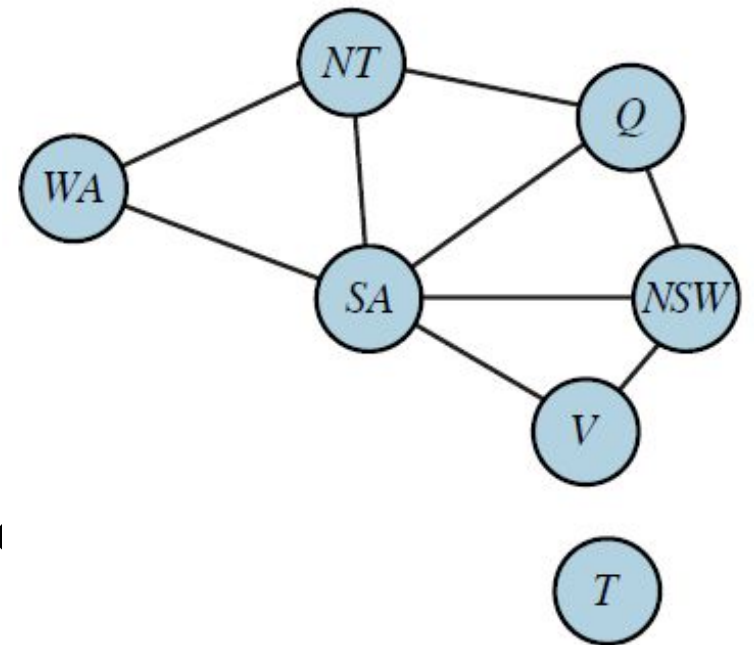
- **Question:** What is the impact of this assignment?

- CSP solver can quickly prune large swathes of the search space that an atomic state-space searcher cannot.



Why Formulate A Problem as CSP?

- Non-CSP search
 - Consider $3^5 = 243$ assignments for the five neighboring variables.
- CSP search
 - Consider $2^5 = 32$ assignments (87% reduction).
- When a partial assignment is found to violate a constraint, we can immediately:
 - Find out which variables violate a constraint.
 - Discard further refinements of the partial assignment.
 - Focus attention on the variables that matter.



Job Shop Scheduling Problem

...The whole job is composed of tasks, and we can **model each task as a variable**, where the **value** of each variable is **the time that the task starts**, expressed as an integer number of minutes. **Constraints** can assert that **one task must occur before another** and specify that **a task takes a certain amount of time to complete...**



Job Shop Scheduling Problem

- Variable, X

- Install axles (front and back).
- Affix all four wheels (right and left, front and back).
- Tighten nuts for each wheel.
- Affix hubcaps for each wheel.
- Inspect the final assembly.
- $X = \{Axle_F, Axle_B, Wheel_{RF}, Wheel_{LF}, Wheel_{RB}, Wheel_{LB}, Nuts_{RF}, Nuts_{LF}, Nuts_{RB}, Nuts_{LB}, Cap_{RF}, Cap_{LF}, Cap_{RB}, Cap_{LB}, Inspect\}$

- Domain, D_i

- The value of each variable is the time that the task starts.

Job Shop Scheduling Problem

- Constraint (or precedence constraint), C
 - Whenever a task T_1 must occur before task T_2 , and task T_1 takes duration d_1 to complete, we add an arithmetic constraint of the form:

$$T_1 + d_1 \leq T_2$$

- Suppose the axles have to be in place before the wheels are put on, and it takes 10 minutes to install an axle, so we write:

$$\text{Axle}_F + 10 \leq \text{Wheel}_{RF}$$

$$\text{Axle}_F + 10 \leq \text{Wheel}_{LF}$$

$$\text{Axle}_B + 10 \leq \text{Wheel}_{RB}$$

$$\text{Axle}_B + 10 \leq \text{Wheel}_{LB}$$

Job Shop Scheduling Problem

- Suppose for each wheel, the wheel must be affixed (1 minute), then tighten the nuts (2 minutes), and finally attach the hubcap (1 minute):

$$\text{Wheel}_{\text{RF}} + 1 \leq \text{Nuts}_{\text{RF}}$$

$$\text{Wheel}_{\text{LF}} + 1 \leq \text{Nuts}_{\text{LF}}$$

$$\text{Wheel}_{\text{RB}} + 1 \leq \text{Nuts}_{\text{RB}}$$

$$\text{Wheel}_{\text{LB}} + 1 \leq \text{Nuts}_{\text{LB}}$$

$$\text{Nuts}_{\text{RF}} + 2 \leq \text{Cap}_{\text{RF}}$$

$$\text{Nuts}_{\text{LF}} + 2 \leq \text{Cap}_{\text{LF}}$$

$$\text{Nuts}_{\text{RB}} + 2 \leq \text{Cap}_{\text{RB}}$$

$$\text{Nuts}_{\text{LB}} + 2 \leq \text{Cap}_{\text{LB}}$$

Job Shop Scheduling Problem

- Suppose there are 4 workers to install wheels, but they have to share one tool that helps put the axle in place, a disjunctive constraint is needed to say that $Axle_F$ and $Axle_B$ must not overlap in time; either one comes first or the other does:

$$(Axle_F + 10 \leq Axle_B) \text{ or } (Axle_B + 10 \leq Axle_F)$$

Job Shop Scheduling Problem

- Suppose the inspection comes last and takes 3 minutes.
- For every variable except Inspect we add a constraint of the form:

$$X + d_x \leq \text{Inspect}$$

$$\text{Axle}_F + 10 \leq \text{Inspect}$$

$$\text{Axle}_B + 10 \leq \text{Inspect}$$

$$\text{Wheel}_{RF} + 1 \leq \text{Inspect}$$

$$\text{Wheel}_{LF} + 1 \leq \text{Inspect}$$

$$\text{Wheel}_{RB} + 1 \leq \text{Inspect}$$

$$\text{Wheel}_{LB} + 1 \leq \text{Inspect}$$

$$\text{Nuts}_{RF} + 2 \leq \text{Inspect}$$

$$\text{Nuts}_{LF} + 2 \leq \text{Inspect}$$

$$\text{Nuts}_{RB} + 2 \leq \text{Inspect}$$

$$\text{Nuts}_{LB} + 2 \leq \text{Inspect}$$

$$\text{Cap}_{RF} + 1 \leq \text{Inspect}$$

$$\text{Cap}_{LF} + 1 \leq \text{Inspect}$$

$$\text{Cap}_{RB} + 1 \leq \text{Inspect}$$

$$\text{Cap}_{LB} + 1 \leq \text{Inspect}$$

Job Shop Scheduling Problem

- Finally, suppose there is a requirement to get the whole assembly done in 30 minutes.
- We can achieve that by limiting the domain of all variables:

$$D_i = \{1, 2, 3, \dots, 27\}$$

- This particular problem is trivial to solve.
- CSPs have been applied to job-shop scheduling problems like this with thousands of variables.

Variations on the CSP Formalism: Domains

- The simplest kind of CSP involves variables that have **discrete, finite domains**.
- Example
 - Map coloring problem
 - Job scheduling problem
 - 8-Queens problem
- A **discrete domain can be infinite**, such as the set of integers or strings.
- With infinite domains, an abstract constraint language must be used instead of enumeration of all possible combinations of values.

Variations on the CSP Formalism: Domains

- Constraint satisfaction problems with **continuous domains** are common in the real-world problems.
- Example
 - The scheduling of certain experiments requires very precise timing of observations, where the start and finish of each observation are continuous-valued variables that must obey a variety of constraints.

Variations on the CSP Formalism: Constraints

□ Unary constraint

- The simplest type of constraint.

- It **restricts the value of a single variable**.

- Example

- In the map-coloring problem, suppose South Australians won't tolerate the color green, and hence that cannot be expressed with a unary constraint:

$$\langle (SA), SA \neq \text{green} \rangle$$

Variations on the CSP Formalism: Constraints

□ Binary constraint

- It **relates two variables**.

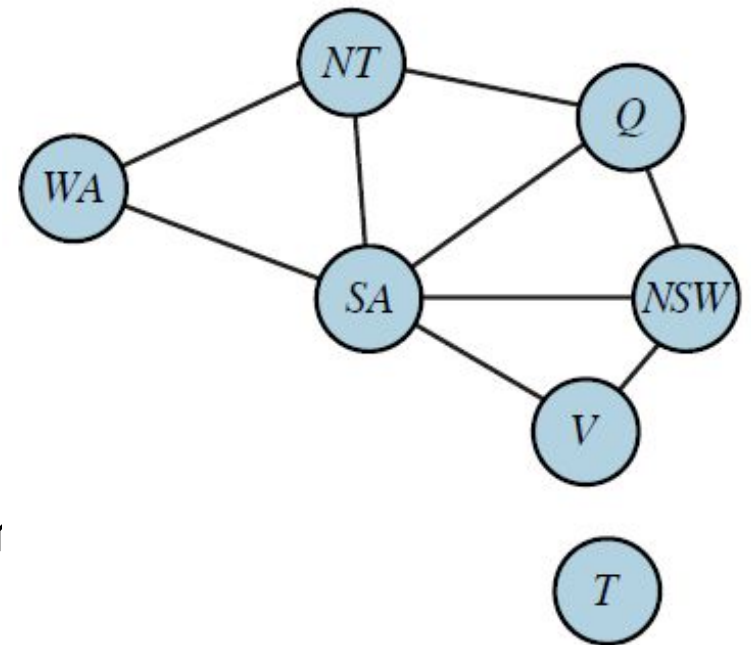
- Example

$\langle SA \neq NSW \rangle$

- It can be represented as a **constraint graph**.

- A **binary CSP** is one with only binary and unary constraints.

□ **Question:** Which constraints provided in the car assen scheduling problem are binary constraints?



Variations on the CSP Formalism: Constraints

□ Ternary constraint

- It is referred to as the **higher-order constraint**.

- Example

- The ternary constraint `Between(X,Y,Z)` can be defined as

- $\langle (X, Y, Z), X < Y < Z \text{ or } X > Y > Z \rangle$

Variations on the CSP Formalism: Constraints

□ Global constraint

- A constraint involving an **arbitrary number of variables**.
- The word "global" is confusing because not all variables are involved.
- One of the most common global constraints is `Alldiff`.
- It says that all variables involved in the constraint must have different values.
- In Sudoku problems, all variables in a row, column, or 3×3 box must satisfy an `Alldiff` constraint.

Binarization of Constraints

- Every finite-domain constraint can be **reduced to a set of binary constraints** if **enough auxiliary variables are introduced**.
- This process is called **binarization of constraints**.
- Transforming a CSP into one with only binary constraints can make the algorithms simpler to some algorithm designers.
- Binarization of constraints can be done via **dual graph transformation** where:
 - One variable for each constraint in the original graph.
 - One binary constraint for each pair of constraints in the original graph that share variables.

Dua Graph Transformation

□ Consider the following CSP formulation:

$$\square X = \{A, B, C\}$$

$$\square D = \{1, 2, 3, 4, 5\}$$

$$\square C = \{\langle (X, Y, Z), X + Y = Z \rangle, \langle (X, Y), X + 1 = Y \rangle\}$$

Binarization of Constraints

□ The dual graph has the following:

□ Variable, $X = \{C_1, C_2\}$

□ Domain for each variable:

□ D_{C_1} = The set of $\{(x_i, y_j, z_k)\}$ tuples from the C_1 constraint in the original problem.

□ D_{C_2} = The set of $\{(x_i, y_j)\}$ tuples from the C_2 constraint in the original problem.

□ Constraint, $C = \langle (C_1, C_2), R \rangle$ where R is a new relation that defines the constraint between C_1 and C_2 : in this case it would be

$$R = \{((1, 2, 3), (1, 2)), ((2, 3, 5), (2, 3))\}$$

Binary Constraint vs. Global Constraint

- However, a global constraint such as `Alldiff` is still preferred over a set of binary constraints because:
 - It is easier and less error-prone to write the problem description using `Alldiff`.
 - It is possible to design special-purpose inference algorithms for global constraints that are not available for a set of more primitive constraints.

Absolute Constraint vs. Preference Constraint

□ Absolute constraint

- The **strictest** type of constraint, in which violation of any of them rules out a potential solution immediately.
- The constraints described so far are absolute constraints.

□ Preference constraint

- A **softer** version of constraint, in which violation of any of them will not rule out a potential solution, but making it a **non-optimal solution**.

Variations on the CSP Formalism: Constraints

- Example: University class-scheduling problem

- **Absolute constraints**

- No professor can teach 2 classes at the same time.

- **Preference constraints**

- Prof. R prefers teaching in the morning.

- Prof. N prefers teaching in the afternoon.

- **Assignment**

- Prof. R teaches two subjects at 8am. (**Not a solution**)

- Prof. R teaches at 10am and Prof. N teaches at 2pm. (**Optimal solution**)

- Prof. R and Prof. N teach at 8am. (**Sub-optimal solution**)

Constraint Optimization Problems

□ Preference constraints

- Preference constraints can often be encoded as costs on individual variable assignments.
- Example
 - Assigning an afternoon slot for Prof. R costs 2 points.
 - Assigning a morning slot for Prof. R costs 1 point.
- With this formulation, CSPs with preferences can be solved with optimization search methods.
- This kind of problem is called a **constraint optimization problem**, or COP.

Constraint Propagation

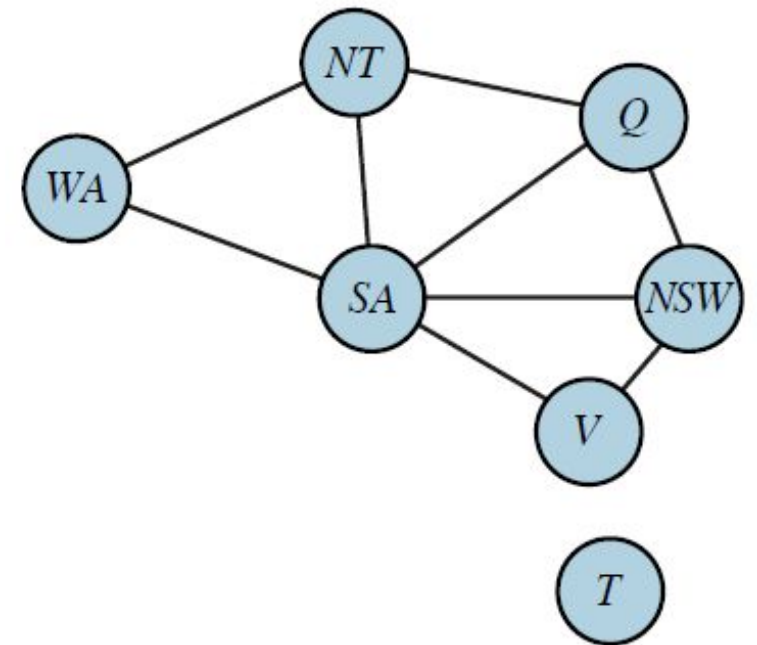
- Non-CSP search
 - With an atomic representation, search algorithm makes progress in only one way.
- CSP search
 - An algorithm can
 - Generate successors by choosing a new variable assignment (search).
 - Do a specific type of **inference** or reasoning called **constraint propagation**.

Constraint Propagation

- **Constraint propagation** makes use of constraints to reduce the number of legal values for a variable, which in turn can reduce the legal values for another variable, and so on.
- This will leave fewer choices to consider when we make the next choice of a variable assignment.
- **Constraint propagation** might be done as a **pre-processing step**, before search starts.
- If this preprocessing can **solve the whole problem**, then no search is required.
- However, it might also be **intertwined with search**.

Local Consistency

- The key idea of constraint propagation is **local consistency**.
- **Local consistency** is the property of CSPs related to the consistency of subsets of variables or constraints.
- Enforcing **local consistency** in a constraint graph can cause inconsistent values to be eliminated.
- There are different types of local consistency:
 - Node consistency
 - Arc consistency
 - Path consistency



Node Consistency

- A **single variable** (corresponding to a node in the CSP network) is **node-consistent** if all the values in the variable's domain **satisfy the variable's unary constraints**.
- Once satisfied, all unary constraints can be eliminated.
- Example: Map Coloring Problem
 - The variable SA starts with domain {red, green, blue}.
 - But the South Australians dislike green.
 - Node consistency can be achieved by eliminating {green}.
 - Consequently, the domain of SA is reduced to {red, blue}.
- A graph is node-consistent if every variable in the network is node-consistent.

Node Consistency

- Since some CSP solvers work with binary constraints only, they expect the user to eliminate the other constraints ahead of time.
- Therefore, we make that assumption for the rest of this chapter, except where noted.

Arc Consistency

- A variable in a CSP is **arc-consistent** if every value in its domain **satisfies the variable's binary constraints**.
- More formally, X_i is arc-consistent with respect to another variable X_j if for every value in the current domain D_i there is some value (the support) in the domain D_j that satisfies the binary constraint on the arc (X_i, X_j) .
- Similarly, X_j is arc-consistent with respect to another variable X_i if for every value in the current domain D_j there is some value (the support) in the domain D_i that satisfies the binary constraint on the arc (X_j, X_i) .
- Thus, a graph is arc-consistent if every variable is arc-consistent with every other variable.

Arc Consistency

- Consider the following CSP where:
 - Variables: $\{X, Y\}$
 - Domains: $\{0, 1, \dots, 9, 10\}$
 - Constraint:
 - $\langle (X, Y), Y = X^2 \rangle$
 - $\langle (X, Y), \{(0, 0), (1, 1), (2, 4), (3, 9)\} \rangle$

Arc Consistency

- To make X arc-consistent with respect to Y, X's domain is reduced to:

$$D_X = \{0, 1, 2, 3\}$$

- If Y is also made arc-consistent with respect to X, then Y's domain becomes:

$$D_Y = \{0, 1, 4, 9\}$$

- Now, the whole CSP is arc-consistent:

- Variables: $\{X, Y\}$

- Domains: $\{D_X = \{0, 1, 2, 3\}, D_Y = \{0, 1, 4, 9\}\}$

- Constraint: $\langle (X, Y), Y = X^2 \rangle$

Reading Homework

- **Artificial Intelligence: A Modern Approach**, 4th Edition, by Stuart Russell and Peter Norvig
 - Chapter 5: Constraint Satisfaction Problems (page 164 to page 170)

The End